

SmartCare v0.1 - Initial Engineering Brief and AI Activity Card

A2 Case Study Stage 1 student resource

SmartCare scenario

SmartCare Community Clinic currently uses spreadsheets and paper records to manage patients and appointments. The client says: 'We need software to help manage patients, practitioners and appointments.' This is not yet a complete specification.

Initial Engineering Brief

1. Problem summary

SmartCare Community Clinic manages patients, practitioners and appointments with spreadsheets and paper records. This causes duplicate bookings, hard-to-find patient records, inconsistent appointment status, poor visibility of practitioner availability, manual cancellations, no reliable appointment history and difficulty producing reports. The clinic wants simple software to manage patients, practitioners and appointments, but the request is not yet a complete specification. The first version should suit a small clinic and not become a complex hospital system. Before designing anything, we need to clarify who will use it, what data must be stored, which booking rules apply and what is in scope.

2. Initial stakeholders

Stakeholder	Possible need
Patients	Easy booking and cancelling, no double-bookings, correct appointment details
Practitioners (GPs)	See their schedule and availability; access patient details and appointment history
Reception/admin staff	Quickly find patients, book and cancel reliably, see consistent appointment status
Clinic management	Basic reports, fewer errors, and a simple system suited to a small clinic

3. Initial features

Feature	Confirmed or provisional?	Why?
Patient record management and search	Confirmed	The client asked to manage patients, and records are currently hard to find.
Appointment booking and cancellation	Confirmed	The client asked to manage appointments; duplicate bookings and manual cancellation are current problems.
Practitioner availability view	Provisional	Availability visibility is a problem, but working hours and availability rules are unknown.
Reports and appointment history	Provisional	Reporting is a stated difficulty, but

		we do not know which reports are needed.
--	--	--

4. Questions for the client

1. Who will use the system day to day (reception, GPs, patients online), and how many users are there?
2. What details must be stored for a patient and for an appointment, and what privacy rules apply to health information?
3. What are the booking rules (appointment length, cancellation notice, allowed statuses, overlapping appointments)?
4. Which reports does management need, and how often?
5. Does existing spreadsheet and paper data need to be moved into the new system for the first version?

5. What we do not yet know

1. Whether patients will book themselves online or only staff will.
2. The legal and privacy requirements for storing patient information.
3. Practitioner working hours, availability rules and how many practitioners there are.

AI Activity Card - Ask, Check, Explain

Before AI

It stores appointments (patient, practitioner, time) in a list of dictionaries. *book_appointment()* checks the patient's name isn't blank and adds the booking. *display_appointments()* prints them all. Some problems are the times are not checked as real dates, there is no cancel or edit, and nothing is saved.

Problems: AI request

Act as a tutor. Explain this code and identify potential problems. Do not provide a complete replacement. Ask me questions that help me reason about the solution.

COPILOT:

appointments is a list that stores each booking as a dictionary. *book_appointment()* checks for an empty patient name and prevents the same practitioner from being booked at the same time. *display_appointments()* prints all bookings, or a message if there are none.

One issue: the first two sample appointments are printed directly, then booked and printed again through the functions. Also, names and time formats aren't fully validated, and bookings aren't saved after the program exits.

What should the program do when a booking conflicts: report the problem, or prompt for a different time?

Evaluate

Suggestion	Useful	Unclear	Incorrect	Out of scope
	Avoid printing the sample appointments twice; use one booking method.		Delete the conflict check so anyone can book the same practitioner at the same time.	
	Validate names and appointment times more carefully.			
	Consider saving appointments so they remain after the program exits.			

--	--	--	--	--

Decide

For each significant suggestion: Accept / Modify / Reject / Keep unverified.

Reject Delete conflict suggestion.

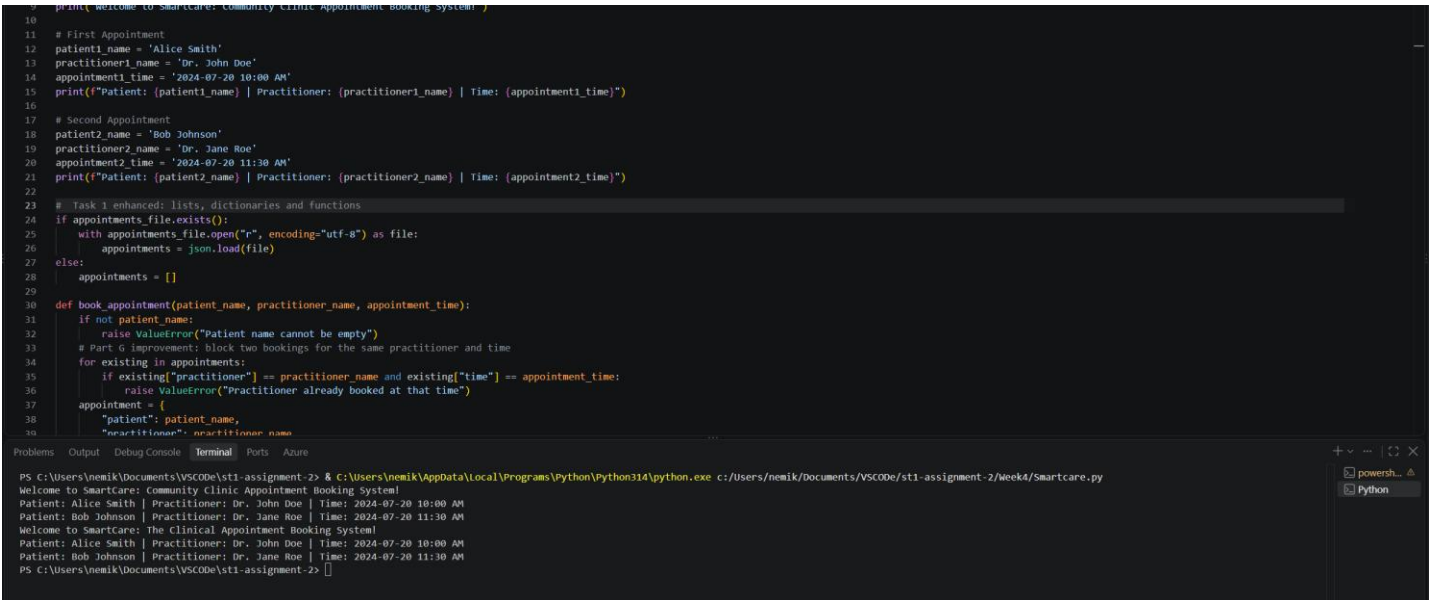
Use the save suggestion

Accept the other 2

Verify

- Run the code
- Test normal input
- Test unusual input
- Compare with requirements
- Ask tutor/peer
- Check documentation

```
9 print("Welcome to SmartCare: Community Clinic Appointment Booking System")
10
11 # First Appointment
12 patient1_name = 'Alice Smith'
13 practitioner1_name = 'Dr. John Doe'
14 appointment1_time = '2024-07-20 10:00 AM'
15 print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
16
17 # Second Appointment
18 patient2_name = 'Bob Johnson'
19 practitioner2_name = 'Dr. Jane Roe'
20 appointment2_time = '2024-07-20 11:30 AM'
21 print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
22
23 # Task 1 enhanced: lists, dictionaries and functions
24 if appointments_file.exists():
25     with appointments_file.open("r", encoding="utf-8") as file:
26         appointments = json.load(file)
27 else:
28     appointments = []
29
30 def book_appointment(patient_name, practitioner_name, appointment_time):
31     if not patient_name:
32         raise ValueError("Patient name cannot be empty")
33     # Part 6 improvement: block two bookings for the same practitioner and time
34     for existing in appointments:
35         if existing["practitioner"] == practitioner_name and existing["time"] == appointment_time:
36             raise ValueError("Practitioner already booked at that time")
37     appointment = {
38         "patient": patient_name,
39         "practitioner": practitioner_name,
40         "time": appointment_time
41     }
42     appointments.append(appointment)
43     with appointments_file.open("w", encoding="utf-8") as file:
44         json.dump(appointments, file)
45
46 # Main execution
47 book_appointment(patient1_name, practitioner1_name, appointment1_time)
48 book_appointment(patient2_name, practitioner2_name, appointment2_time)
49
50 # Print final appointments
51 with appointments_file.open("r", encoding="utf-8") as file:
52     appointments = json.load(file)
53     for appointment in appointments:
54         print(f"Patient: {appointment['patient']} | Practitioner: {appointment['practitioner']} | Time: {appointment['time']}")
55
56 # End of program
57 print("Program completed successfully.")
```



Explain

Can I explain the final code without reading the AI response? What do I still need to understand?

I can explain it, however the new save suggestion I still don't understand.